

ID	Name	Email	Creatad_At
1	1801	1800	Ferde
2	28003	2001	Beaded_At
3	23905	1932	Lenrei
3	28000	1834	Henrew
4	2203	280	Fond
5	1807	1608	Creatd_ade
6	2301	1583	Reoved_At
7	240	140	Lererer
8	2803	1802	Fond
9	1403	1604	Rede

Introdução ao Modelo Relacional



Eduardo Ogasawara

eogasawara@cefet-rj.br | <https://eic.cefet-rj.br/~eogasawara>



Conceitos Fundamentais

Uma relação é definida por seu **schema**, que lista os atributos $A_1, A_2, \dots, A_n$ que a compõem. A notação formal é:
 $R = (A_1, A_2, \dots, A_n)$.



Exemplo: Schema instructor

instructor = (ID, name, dept_name, salary)

Instância e Tuplas

Uma **instância de relação** $r(R)$ representa os valores atuais da relação em um determinado momento, expressos como uma **tabela**.

Cada linha da tabela é chamada de **tupla** e contém valores específicos para todos os atributos definidos no schema.

Domínio de Atributos

O conjunto de valores permitidos para cada atributo é chamado de **domínio**. Por exemplo, o domínio de *salary* são números positivos, enquanto o de *dept_name* são strings de texto.

Valor Null

O valor especial **null** é membro de todos os domínios e indica que o valor é desconhecido ou não aplicável. Por exemplo, um instrutor pode não ter e-mail registrado.

Atomicidade de Valores

Os valores de atributos são exigidos como **atômicos**, ou seja, indivisíveis. Cada valor deve ser uma unidade única e não pode ser decomposto em partes menores.

Complicações com Null

O valor null causa complicações em muitas operações de banco de dados, exigindo lógica especial de três valores (verdadeiro, falso, desconhecido). Agregações também devem decidir como tratar valores ausentes.

Conceito Fundamental

Uma propriedade crucial das relações é que a **ordem das tuplas é irrelevante**. As tuplas podem ser armazenadas em qualquer ordem arbitrária sem afetar o significado da relação. Diferentemente de arrays ou listas, a identidade de uma tupla é determinada por seus valores, não por sua posição.

Exemplo: Relação instructor com tuplas não ordenadas

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
33456	Gold	Physics	87000

Observe que as mesmas tuplas podem aparecer em qualquer ordem na tabela. O significado dos dados permanece idêntico independentemente de como as linhas estão organizadas fisicamente.

Database Schema

O **schema do banco de dados** representa a estrutura lógica completa do banco de dados, definindo relações, atributos, tipos de dados e restrições. Permanece relativamente estável ao longo do tempo.

Exemplo de Schema

instructor (ID, name, dept_name, salary)

Este schema define a estrutura da relação instructor com quatro atributos específicos.

Database Instance

A **instância do banco de dados** é um snapshot dos dados em um determinado instante no tempo. Ao contrário do schema, a instância muda frequentemente conforme dados são inseridos, atualizados ou excluídos.

Exemplo de Instância

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
33456	Gold	Physics	87000

Superkey

Seja $K \subseteq R$. **K** é uma **superkey** de R se os valores de K são suficientes para identificar uma tupla única de cada relação possível $r(R)$. Por exemplo, tanto $\{ID\}$ quanto $\{ID, name\}$ são superkeys de *instructor*.

Primary Key

Uma das candidate keys é selecionada para ser a **primary key** da relação. A escolha prioriza chaves simples, estáveis e com significado claro no domínio do problema.

Candidate Key

Uma superkey K é uma **candidate key** se é minimal — nenhum subconjunto próprio de K é também uma superkey. Assim, $\{ID\}$ é candidate key de *instructor*, mas $\{ID, name\}$ não é.

Foreign Key

Uma **foreign key** garante que um valor em uma relação referenciante apareça na primary key de outra relação referenciada. Por exemplo, *dept_name* em *instructor* referencia a relação *department*.

Diagrama de Esquema para Banco de Dados Universitário

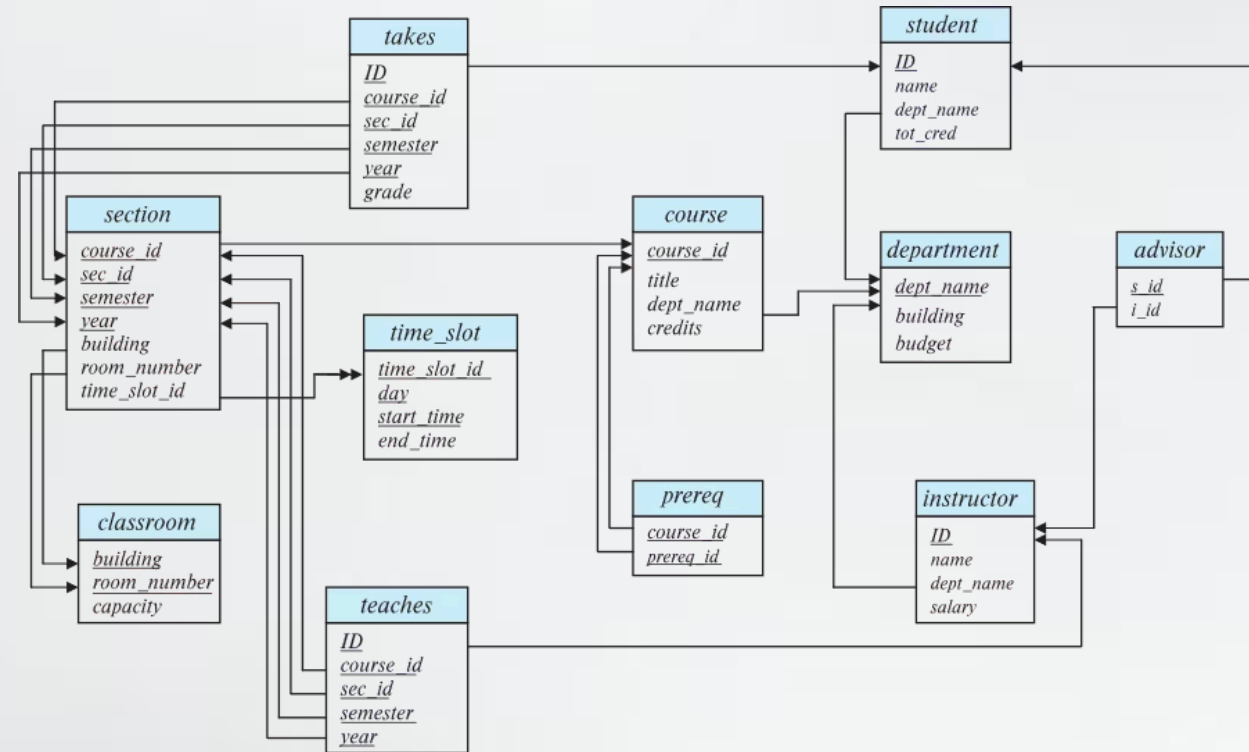
O diagrama de esquema fornece uma representação visual completa das relações no banco de dados e seus relacionamentos. Cada caixa representa uma relação com seus atributos, e as setas indicam foreign keys que conectam as relações.

Componentes Principais

- **Relações:** Cada caixa representa uma tabela no banco de dados
- **Atributos:** Listados dentro de cada caixa
- **Primary Keys:** Sublinhados para identificação rápida
- **Foreign Keys:** Indicadas por setas conectando relações

Interpretação

Este diagrama mostra como diferentes entidades do sistema universitário - instrutores, departamentos, cursos, estudantes - estão interconectadas por meio de foreign keys, formando uma rede coesa de informações relacionadas.



Linguagens Procedurais vs. Declarativas

Linguagens **procedurais** especificam como obter os dados, passo a passo. Linguagens **declarativas** especificam quais dados queremos, deixando o sistema decidir como obtê-los.

Linguagens "Puras"

Três linguagens relacionais puras formam a base teórica: **Álgebra Relacional** (procedural), **Cálculo Relacional de Tuplas** e **Cálculo Relacional de Domínio**. As três são equivalentes em poder computacional.

Foco

Concentraremos nosso estudo na **álgebra relacional**, base fundamental para entender operações de banco de dados.

Características da Álgebra Relacional

- Consiste em 6 operações básicas
- Forma a base de linguagens SQL modernas

Select (σ)

Seleciona tuplas que satisfazem um predicado específico. Filtra linhas da relação com base em uma condição.

Union ($\cup$)

Combina tuplas de duas relações, eliminando duplicatas. Requer que as relações sejam compatíveis.

Cartesian Product ($\times$)

Combina cada tupla da primeira relação com cada tupla da segunda. Cria todas as combinações possíveis.

Project (Π)

Retorna a relação com certos atributos removidos. Seleciona colunas específicas da relação.

Set Difference ($-$)

Retorna tuplas que estão na primeira relação mas não na segunda. Realiza subtração de conjuntos.

Rename (ρ)

Renomeia uma relação e/ou seus atributos. Útil para evitar ambiguidades em operações complexas.

Definição e Notação

A operação **select** seleciona tuplas que satisfazem um predicado específico. A notação formal é:

$$\sigma_p(r)$$

onde p é chamado de **predicado de seleção** e r é a relação.

❏ O predicado pode incluir comparações ($=$, $\neq$, $<$, $>$, $\leq$, $\geq$) e conectivos lógicos ($\wedge$ para AND, $\vee$ para OR, $\neg$ para NOT).

Exemplo Prático

Selecionar tuplas da relação *instructor* onde o instrutor está no departamento "Physics":

Consulta:

$$\sigma_{\text{dept_name}=\text{"Physics"}}(\text{instructor})$$

Resultado:

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Relação Completa

Contém todos os atributos definidos no schema

Aplicação do Project

$\Pi_{A_1, A_2, \dots, A_k}(r)$

Relação Resultante

Apenas k colunas selecionadas permanecem

Definição

Uma operação unária que retorna sua relação argumento com certos atributos removidos. A notação $\Pi_{A_1, A_2, \dots, A_k}(r)$ seleciona apenas as colunas listadas, apagando as demais.

Propriedade Importante

Linhas duplicadas são removidas do resultado, pois relações são conjuntos matemáticos. Isso garante que o resultado contenha apenas tuplas distintas.

Objetivo

Eliminar o atributo *dept_name* da relação *instructor*, mantendo apenas *ID*, *name* e *salary*.

Consulta em Álgebra Relacional

$\Pi_{ID, name, salary}(instructor)$

A operação remove a coluna *dept_name* e elimina quaisquer linhas duplicadas do resultado.

Resultado da Operação

ID	name	salary
22222	Einstein	95000
12121	Wu	90000
32343	El Said	60000
45565	Katz	75000
33456	Gold	87000

A tabela resultante contém apenas as três colunas especificadas na operação project.

Passo 1: Produto Cartesiano

`instructor` $\times$ `teaches` associa cada tupla de `instructor` com cada tupla de `teaches`

Passo 2: Aplicar Select

$\sigma_{\text{instructor.id=teaches.id}}(\text{instructor} \times \text{teaches})$

O Desafio do Produto Cartesiano

O produto cartesiano cria **todas as combinações possíveis** entre duas relações — a maioria sem sentido, representando associações entre instrutores e cursos que eles nunca ensinaram.

Problema Identificado

A maioria das linhas resultantes contém informações sobre instrutores que NÃO ensinaram um curso específico

Resultado Final

Apenas tuplas que relacionam instrutores aos cursos que eles efetivamente ensinaram

A Solução: Join

Aplicamos uma condição de seleção

$\sigma_{\text{instructor.id=teaches.id}}(\text{instructor} \times \text{teaches})$ para obter apenas tuplas relevantes. Essa expressão tão comum é chamada de **join**, uma das operações mais importantes em bancos de dados relacionais.

Problema de Exemplo

Encontrar instrutores do departamento de Physics com salário maior que 90.000.

Consulta 1: Seleção Combinada

$\sigma_{\text{dept_name}=\text{"Physics"} \wedge \text{salary} > 90000}(\text{instructor})$

Aplica **ambas as condições simultaneamente** em uma única operação.

Consulta 2: Seleções Sequenciais

$\sigma_{\text{dept_name}=\text{"Physics"}}(\sigma_{\text{salary} > 90000}(\text{instructor}))$

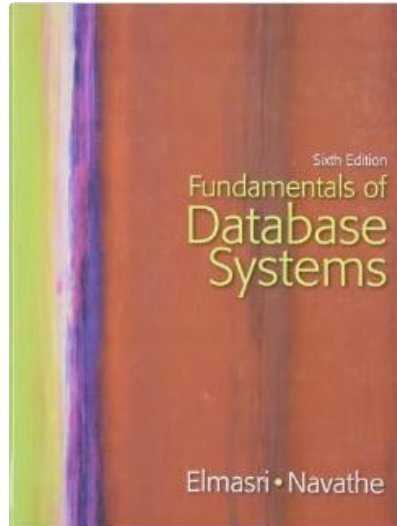
Aplica **duas seleções em sequência**: primeiro por salário, depois por departamento.

Equivalência

As duas consultas **não são idênticas** em estrutura, mas são **equivalentes** — produzem o mesmo resultado em qualquer banco de dados possível.

 O SGBD pode escolher a representação mais eficiente para executar a consulta, independentemente de como ela foi escrita — essencial para otimização de consultas.

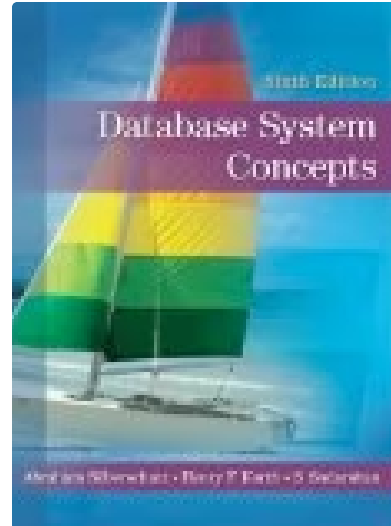
Referências



Elmasri & Navathe

Fundamentals of Database Systems

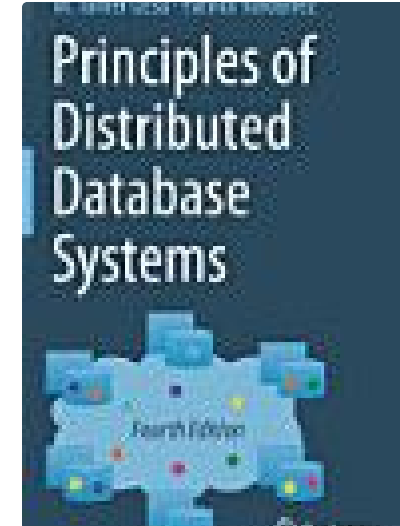
Pearson, 2016



Korth, Sudarshan & Silberschatz

Database System Concepts

McGraw-Hill, 2019



Özsu & Valduriez

Principles of Distributed Database Systems

Springer Nature, 2019